



파머완 1장. 파이썬 기반의 머신러닝과 생태계 이해

1팀 김선우, 박채윤, 이성연

목차

#01 넘파이

- 1-1 넘파이 소개
- 1-2 넘파이 ndarray
- 1-3 ndarray의 데이터 타입
- 1-4 ndarray 편리하게 생성하기, 크기 바꾸기
- 1-5 ndarray 데이터 세트 선택 : 인덱싱, 슬라이싱
- 1-6 행렬의 정렬 : sort(), argsort()
- 1-7 선형대수 연산 : 행렬 내적, 전치 행렬

#02 판다스

- 2-1 판다스 소개
- 2-2 DataFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환
- 2-3 DataFrame의 칼럼 데이터 세트 생성과 수정
- 2-4 DataFrame 데이터 삭제
- 2-5 Index 객체



01. 넘파이(Numpy)



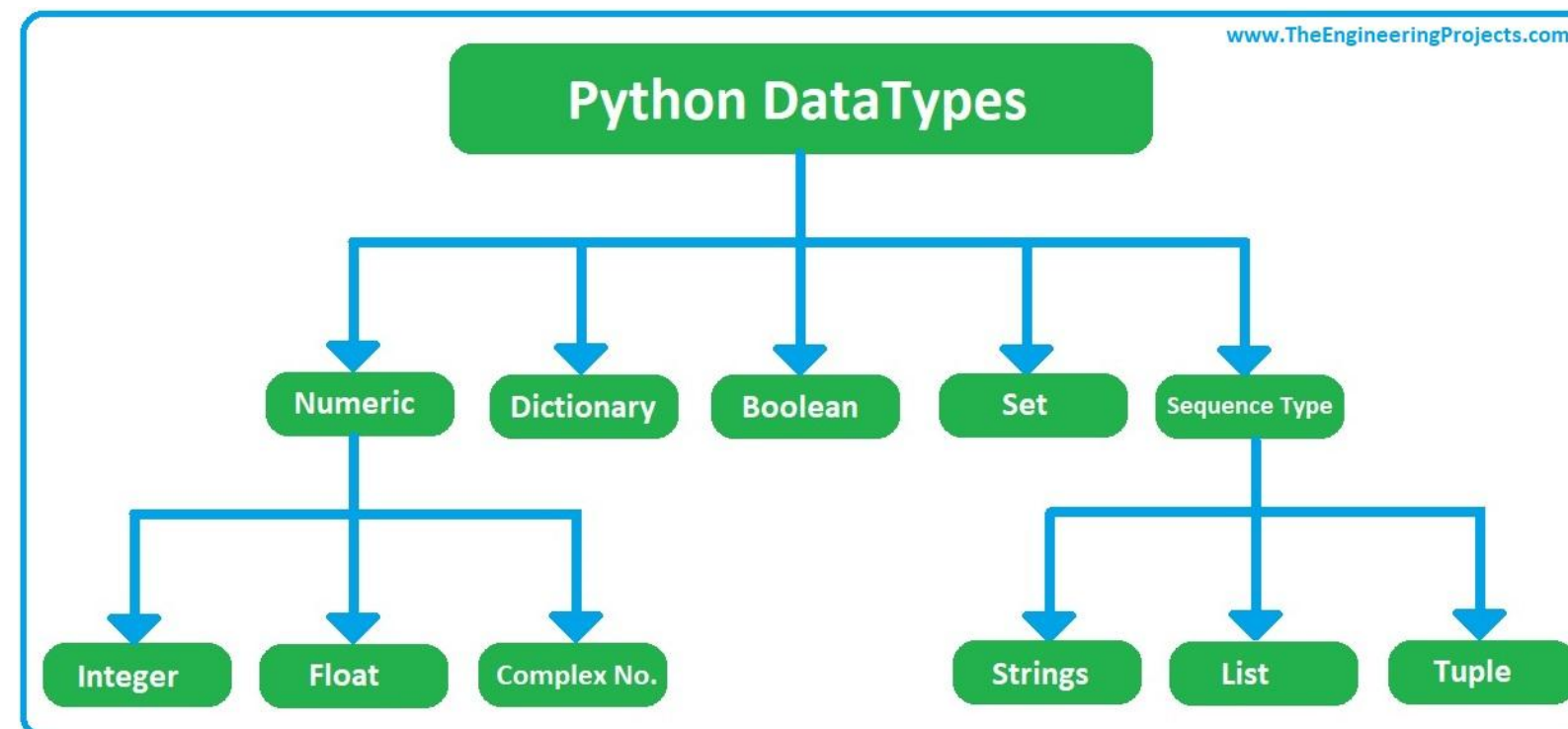
넘파이 ndarray

1. 넘파이 모듈 импорт

Import numpy as np

약어로 모듈 표현 편리!

2. 넘파이 기반 데이터 타입이 ndarray : 다차원 배열 생성, 연산 수행



넘파이 ndarray

#1 NumPy 배열은 **차원(dimension)**과 **형태(shape)**라는 두 개념으로 구조를 표현

차원(ndim) → 배열이 몇 차원인지

형태(shape) → 각 차원의 데이터 개수 = (행, 열)의 구조

```
import numpy as np
```

<code>array1 = np.array([1, 2, 3])</code>	1차원 배열
---	--------

<code>array2 = np.array([[1, 2, 3], [2, 3, 4]])</code>	2차원 배열
--	--------

<code>array3 = np.array([[1, 2, 3]])</code>	2차원 배열(1행 3열)
---	---------------

#array1과 array3는 데이터는 같으나 차원이 다름!

ndarray 주요 속성

#1 NumPy의 핵심 데이터 구조: 다차원 배열

속성	설명
shape	배열 크기 반환
ndim	배열 차원
dtype	데이터 타입
astype()	데이터 타입 변환

```
print(array1.shape)  (3,)      3개의 데이터
print(array2.shape)  (2,3)     2행 3열
print(array3.shape)  (1,3)     1행 3열
```

데이터가 같아도 shape이 다르면 다른 배열!

ndarray 주요 속성

#1 NumPy의 핵심 데이터 구조: 다차원 배열

속성	설명
shape	배열 크기 반환
ndim	배열 차원
dtype	데이터 타입
astype()	데이터 타입 변환

```
print(array1.ndim)    array1: 1차원    3개의 데이터
print(array2.ndim)    array2: 2차원    2행 3열
print(array3.ndim)    array3: 2차원    1행 3열
```

ndarray 주요 속성

#1 NumPy의 핵심 데이터 구조: 다차원 배열

속성	배열	데이터 타입
shape	[1,2,3] Int – 정수형	int
ndim	[1.0,2.0,3.0] Float – 실수, 소수점 표현	float
	['a','b','c'] String- 문자형	string
dtype	데이터 타입	NumPy 배열은 하나의 배열 안에서 동일한 데이터 타입만 저장함
astype()	데이터 타입 변환	

```
import numpy as np
array1 = np.array([1,2,3])
print(array1.dtype)
```

Int64
➔ 데이터 타입을 64bit 정수형(int64)로 저장한다는 뜻
➔ 혼합된 경우에는 더 큰 데이터 타입으로!

ndarray 주요 속성

#1 NumPy의 핵심 데이터 구조: 다차원 배열

혼합시 큰 데이터 타입으로 통일한다했는데,
작은거로 유지하여 용량을 낮추고 싶을 때 사용함

속성	설명
shape	배열 크기 반환
ndim	배열 차원
dtype	데이터 타입
astype()	데이터 타입 변환

```
array1 = np.array([1,2,3])
array_float = array1.astype('float64')
print(array_float)
print(array_float.dtype)
```

[1. 2. 3.]
Float64
= 정수배열을 실수배열로 변환시킴

ndarray의 데이터 타입

#1 숫자값, 문자값, 불값 등 모두 가능
!! Nddarray 내 같은 데이터 타입만 가능 (ex. Int형 + float형 불가능)

#2 데이터 타입 확인하기
: dtype 속성

```
Print(array1.dtype)
```

#3 다른 데이터 유형 섞인 리스트 -> ndarray 변환 시
: 크기가 더 큰 데이터 타입으로 일관 형반환

#4 데이터 타입 변경
: astype() 메서드

```
Array2 = array1.astype('float64')  
Array2 = array1.astype('int32')
```

속성 vs 메서드?

속성 = 객체가 가지고 있는 “값”
그냥 꺼내 읽는 것

메서드 = 객체가 가지고 있는 “함수”
해당 객체에 어떤 동작을 시키는 것이라 () 실행 !

ndarray 편리하게 생성하기

#1 NumPy의 핵심 데이터 구조: ndarray는 다차원 배열 생성, 연산수행을 가능케 함

함수	기능
arrange	0 ~ (인자-1)까지의 배열 생성
reshape	변환 원하는 크기를 인자로 입력, Ndarray 특정 차원 및 크기로 변환
zeros	튜플 형태의 shape 값 입력, 모든 값을 0으로 채움
ones	튜플 형태의 shape 값 입력, 모든 값을 1로 채운 ndarray 반환

배열 생성 함수 – array, arrange

#1 NumPy의 핵심 데이터 구조: ndarray는 다차원 배열 생성, 연산수행을 가능케 함

함수	기능
Np.array	배열 생성
Np.arrange	일정 간격 배열 생성

Array() 함수는 파이썬 리스트를 NumPy배열 , ndarray로 변환함

```
import numpy as np
```

```
array1 = np.array([1,2,3])
```

```
array2 = np.array([[1,2,3],[2,3,4]])
```

배열 생성 함수 – array, arrange

#1 NumPy의 핵심 데이터 구조: ndarray는 다차원 배열 생성, 연산수행을 가능케 함

함수	기능
Np.array	배열 생성
Np.arange	일정 간격 배열 생성 -> 파이썬의 range와 유사한 함수

array1
[0 1 2 3 4 5 6 7 8 9]
#0부터 10개 세고 stop

Array2
[1 3 5 7 9]
#1부터 10까지 2개씩 건너뛰어서 셈

np.arange(start, stop, step)

시작값, 끝 값, 증가간격

Stop이 기본 설정이므로, 다른 것을 쓰지 않으면 0부터 시작해 n개만큼 세는 구조

array1 = np.arange(10)

array2 = np.arange(1,10,2)

arrange()

다음 코드의 출력 결과를 작성하시오

```
import numpy as np
```

```
array1 = np.arange(2, 10, 2)
```

```
print(array1)
```

```
# start =2
```

```
stop = 10 (포함되지 않음)
```

```
step = 2
```

```
# 정답
```

```
[2 4 6 8]
```

reshape()

#1 NumPy 배열의 차원(shape)을 변경하는 함수

즉, 데이터 개수는 유지하면서 배열의 구조만 변경

`array.reshape(행, 열)`

```
import numpy as np  
array1 = np.arange(6)  
array2 = array1.reshape(2, 3)  
print(array2)
```

```
[[0 1 2]  
 [3 4 5]]
```

`Reshape(2, 3)`, `reshape(6, 1)`
`Reshape(6, 1)` 가능

`Reshape(3, 3)` 불가능
→ 데이터의 개수가 다름

zeros(), ones()

#1 튜플 형태의 shape값을 입력할 때, 값을 채우는 함수

배열(또는 행렬)을 빠르게 초기화하려고 배열의 구조만 변경

`array.reshape(행, 열)`

```
import numpy as np
a = np.zeros((3,3))
print(a)
```

[[0. 0. 0.]
[0. 0. 0.]
[0. 0. 0.]

➔ 0으로 다 채운 `zeros()`

```
b = np.ones((2,4))
print(b)
```

[[1. 1. 1. 1.]
[1. 1. 1. 1.]

➔ 1로 다 채운 `ones()`

ndarray 데이터 세트 선택하기 - 인덱싱, 슬라이싱

#1 단일 값 추출 - 1차원

: 원하는 위치의 인덱스 값 [] 안 입력

```
value1 = array[6]
```

: 인덱스 값 -1
맨 뒤의 값

응용 - 뒤에서 두 번째 값

```
value2 = array[-2]
```

#2 단일 값 변경

: 단일 인덱스 사용해 변수 선언처럼 간단하게 변경 가능

```
array[6] = 1  
array[6] = 4
```

#1-1 단일 값 추출 - 2차원

: 원하는 위치의 인덱스 값 [] 안 입력

```
value3 = array[ 0, 0]  
value4 = array[ 0, 1]
```

열 : axis 1

행 : axis 0

	COL 0	COL 1	COL 2
ROW 0	Index (0,0) 1	(0,1) 2	(0,2) 3
ROW 1	(1,0) 4	(1,1) 5	(1,2) 6
ROW 2	(2,0) 7	(2,1) 8	(2,2) 9

※ 다차원 array는 축 기반 연산, axis 생략 시 **행(axis0)** 의미

ndarray 데이터 세트 선택하기 - 인덱싱, 슬라이싱

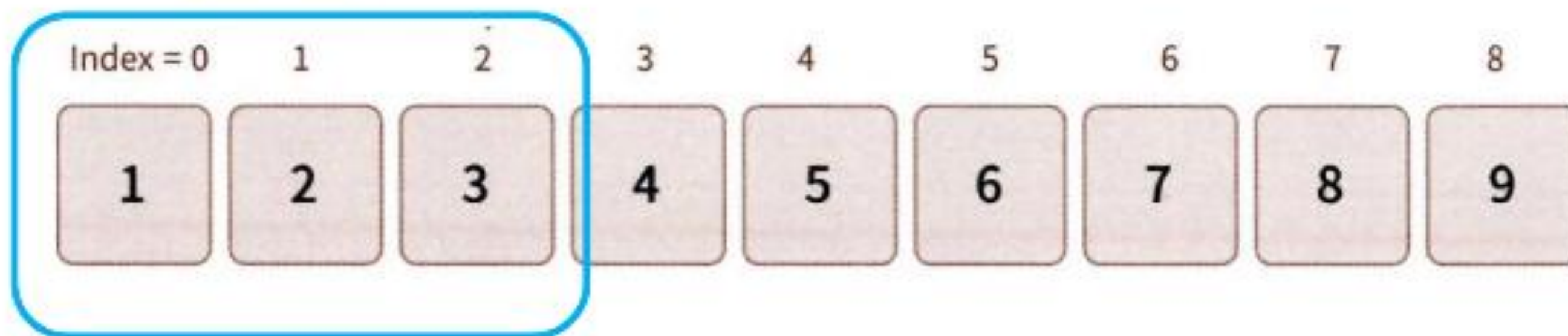
#3-1 슬라이싱 - 1차원

: “:” 기호 사이 시작 인덱스와 종료 인덱스 표시.
-> 시작 인덱스 ~ (종료 인덱스-1) 까지의 ndarray 반환

: 인덱스 생략 가능!

- 시작 인덱스 생략 : 맨 처음 인덱스 0 간주
- 종료 인덱스 생략 : 맨 마지막 인덱스 간주
- 모두 생략 : 0~ 마지막

```
value1 = array1[0:3]
```



#3-2 슬라이싱 - 2차원

: 1차원과 유사, “,” “ ” 로 행 인덱스와 칼럼 인덱스 구분

ndarray 데이터 세트 선택하기 - 인덱싱, 슬라이싱

#3-2 슬라이싱 - 2차원

: 1차원과 유사, “.” 로 행 인덱스와 칼럼 인덱스 구분

1	2	3
4	5	6
7	8	9

array[0:2, 0:3]

1	2	3
4	5	6
7	8	9

array[1:3, :]

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Indexing

팬시 인덱싱

: 리스트나 ndarray로 인덱스 집합 지정
→ 해당 위치 ndarray 반환

1	2	3
4	5	6
7	8	9

`array[[0, 1], 2]`
팬시 인덱싱 단일 인덱싱

↓
인덱스 (0,2), (1,2)
[3, 6] 반환

불린 인덱싱

: 조건 필터링 + 검색
인덱스 지정 [] 내에 조건문 기재

`array[ array > 5 ]`

결과: [6 7 8 9]

1. 조건식 부분 `array > 5` 에서 불린 배열이 생성됨

Index = 0 1 2 3 4 5 6 7 8
[1 2 3 4 5 6 7 8 9]
[F F F F F T T T T]

2. []는 True 값이 있는 위치 인덱스 해당 값을 자동 변환해 반환
(False 값은 무시)

행렬의 정렬을 돕는 sort(), argsort()

sort

이름	기능	특징	원래 행렬	반환값	차순	기타
np.sort()	행렬 정렬	넘파이에서 호출	원본 행렬 변경 X	정렬된 행렬	기본 오름차순	내림차순 정렬 시 [::-1] 사용
ndarray.sort()		행렬 자체에서 호출	정렬됨 (변경됨)	None		

#2

기능: 배열의 요소를 오름차순으로 정렬한 결과를 반환

특징: 원본 배열을 변경하지 않고 정렬된 복사본을 반환하는 경우가 많음 !

(np.sort(arr)).

행렬의 정렬을 돕는 sort(), argsort()

argsort

: 행렬 정렬 후, 기존 원본 행렬의 인덱스 반환
+ 내림차순 정렬 시[::-1] 사용

np.argsort(A) 호출

원본 행렬 인덱스 0 1 2 3

원본 행렬 A [3 1 9 5]

정렬된 행렬의 원본 행렬 인덱스 추출

정렬된 행렬의
원본 행렬 인덱스 1 0 3 2

정렬 행렬 [1 3 5 9]

결과 : [1 0 3 2]

```
array1 = np.array( [ 3, 1, 9, 5] )  
array_sort = np.argsort( array1 )
```

```
print(array_sort)
```

np.argsort() RETURNS THE INDEX VALUES
IN AN ORDER THAT WOULD SORT THE INPUT ARRAY

20	10	40	50	30
----	----	----	----	----

Index: 0 1 2 3 4

np.argsort()

1	0	4	2	3
---	---	---	---	---

argsort()

sort랑 동일하게 정렬인데, 인덱스 값 반환

역순 정렬이나 기준 축 변경 또한 가능

```
1 np.random.seed(42)
2 a = np.random.randint(0, 10, 12).reshape([3,4])
3 a # 2차원 배열 다시 복구하고
```

```
array([[6, 3, 7, 4],
       [6, 9, 2, 6],
       [7, 4, 3, 7]])
```

```
1 np.argsort(a) # 인덱스 값으로 정렬해줌
2               # ex. 1-3-0-2번 순으로 숫자가 작음
```

```
array([[1, 3, 0, 2],
       [2, 0, 3, 1],
       [2, 1, 0, 3]], dtype=int64)
```

```
1 np.argsort(a)[::-1] # 이것도 역순정렬 가능
```

```
array([[2, 0, 3, 1],
       [1, 3, 0, 2],
       [3, 0, 1, 2]], dtype=int64)
```

```
1 np.argsort(a, axis=0) # 행 기준(=세로로)도 가능
```

```
array([[0, 0, 1, 0],
       [1, 2, 2, 1],
       [2, 1, 0, 2]], dtype=int64)
```

행렬 연산

이름	기능	코드 예시	결과 예시
<code>np.dot()</code>	인수로 받은 두 행렬의 내적(행렬곱) 수행	<pre>A = np.array([1,2,3], [4,5,6]) B = np.array([7,8],[9,10],[11,12]) dot_product = np.dot(A, B) print(dot_product)</pre>	<pre>[[58 64] [139 154]]</pre>
<code>np.transpose()</code>	인수로 받은 행렬을 전치	<pre>A = np.array([1,2], [3,4]) transpose = np.transpose(A) print(transpose)</pre>	<pre>[[1 3] [2,4]]</pre>

02. 판다스 part1



판다스 소개

판다스와 넘파이의 차이?

판다스는 데이터 처리를 위해 존재하는 가장 인기 있는 파이썬 '라이브러리'

기능 모음집
원하는 시점에 불러서 씀

2차원 데이터를 효율적으로 가공/처리할 수 있도록 함

행과 열로 이루어진 데이터를 말함

넘파이와 비교 1.

판다스는 넘파이보다 고수준 API로 더 유연함.

함수·메서드들의 모음

-> 더 편리한 데이터 핸들링 가능

넘파이와의 비교 2.

파이썬의 리스트, 컬렉션, 넘파이 등의 내부 데이터 + csv파일도 모두 'DataFrame'으로 변환

-> 가공/분석이 쉬움

data frame :

	survived	pclass	sex	age
0	0	3	male	22.0
1	1	1	female	38.0
2	1	3	female	26.0

Index:

개별 데이터를
고유하게 식별하는
Key 값

Series:

칼럼이 하나 뿐인 데이터 구조체
*DataFrame은 여러 개의 Series로
이뤄졌다고 할 수 있음.

*Series와 DataFrame은 모두
Index를 key 값으로 가지고 있음.

판다스 소개

```
# 판다스 시작은
Import pandas as pd      # 동일하게 약어 사용!
```

판다스 데이터 로딩 API:

* 실무에서는 거의 read_csv()만 사용
* sep을 활용해서 범용성 갖출

함수명	기본 구분자	특징 / 용도	예시
read_csv()	coma(,)	가장 범용적, sep로 다양한 구분자 지정 가능	pd.read_csv("data.csv") pd.read_csv("data.txt", sep="\t")
read_table()	탭(\t)	사실상 read_csv(sep="\t")와 동일	pd.read_table("data.txt")
read_fwf()	없음(고정폭)	고정 폭 열 파일 처리	pd.read_fwf("data.fwf")

판다스 소개

판다스 dataframe 탐색과 요약 :

기능	메서드 / 속성	설명
일부 출력	.head(n)	앞의 n개 행 출력 (기본 5개)
크기 확인	.shape	(행, 열) 튜플 반환
데이터 구조 확인	.info()	행/열 수, 데이터 타입, Null 개수
수치형 데이터 분포 확인	.describe()	숫자형 칼럼의 평균, 표준편차, 분포
카테고리형 데이터 분포 확인	.value_counts()	칼럼별 데이터 값의 분포도를 Series 객체로 반환 맨 왼쪽: 인덱스값 오른쪽: 데이터값
단일 칼럼	df['칼럼명']	Series 반환 (인덱스+값)

카테고리형 데이터 분포의 경우
Null 포함 여부는 dropna인자로 판단
기본은 null을 불포함
포함하고 싶으면 dropna=false로

DataFrame과 상호 변환

dataframe 에는 리스트, ndarray, 딕셔너리

csv를 불러올 수도 있으나 기본적으로는 파이썬의 리스트, 딕셔너리, 그리고 넘파이 ndarray등으로 변환함

1. 리스트와 ndarray를 2차원인 DataFrame으로 변환하는 경우
-> 1차원인지, 2차원인지에 따라 칼럼 개수만 조정해 주면 나머지는 동일

3개의 칼럼명이 필요함.

```
col_name2=['col1', 'col2', 'col3']
```

칼럼을 필요한 개수만큼의 리스트로 지정

리스트인 경우

```
df_list2 = pd.DataFrame(list2, columns=col_name2)
```

ndarray인 경우

변환대상

DataFrame에서 칼럼 명

```
df_array2 = pd.DataFrame(array2, columns=col_name2)
```

Index DataFrame화된
리스트, ndarray

	Col 1
0	3
1	2
2	1

2. 딕셔너리를 DataFrame으로 변환하는 경우
-> 딕셔너리의 키(Key)는 칼럼명으로, 딕셔너리의 값(Value)은 칼럼 데이터로 자동 매핑

Key는 문자열 칼럼명으로 매핑, Value는 리스트 형(또는 ndarray) 칼럼 데이터로 매핑

```
dict = {'col1':[1, 11], 'col2':[2, 22], 'col3':[3, 33]}
```

```
df_dict = pd.DataFrame(dict)
```

```
print('딕셔너리로 만든 DataFrame:\n', df_dict)
```

Index	col1	col2	col3
0	3	4	7
1	2	5	8
2	1	6	9

DataFrame화된
리스트, ndarray

DataFrame과 상호 변환

#2 DataFrame -> 리스트/ndarray/딕셔너리

머신러닝 패키지가 기본 데이터 형으로 넘파이 ndarray를 많이 사용하여 **반대로** 변환하는 경우가 생김

1. 넘파이 ndarray로 변환하기 -> .values

```
array3 = df_dict.values
```

변환대상 DataFrame

2. 리스트로 변환하기 -> .tolist()

```
list3 = df_dict.values.tolist()
```

*2차원을 변환해야 하기 때문에 넘파이의 ndarray로 변환하는 과정이 필요한 것.

-> 따라서 1차원만 변환할 때는 바로 .tolist()사용 가능

Ex) list1 = df_series.tolist()

3. 딕셔너리로 변환하기 -> .to_dict()

```
dict3 = df_dict.to_dict('list')
```

*데이터 타입 부분에 'list' 라고 지정해주면 딕셔너리의 값이 리스트형으로 나옴.

```
df_dict.to_dict() 타입: <class 'dict'>  
{ 'col1': [1, 11], 'col2': [2, 22], 'col3': [3, 33]}
```

DataFrame의 컬럼 데이터 세트 생성과 수정

1. 새로운 컬럼 생성:

[] 연산자를 이용해 데이터프레임에 새로운 컬럼을 추가할 수 있음

2. 기존 컬럼을 이용한 새로운 컬럼 생성:

기존 컬럼 데이터를 활용하여 새로운 컬럼을 만들 수 있음

3. 기존 컬럼 값 수정

컬럼에 새로운 값을 할당하여 기존 데이터를 일괄적으로 업데이트 할 수 있음

DataFrame의 컬럼 데이터 세트 생성과 수정

새로운 컬럼 생성	기존 컬럼을 이용한 새로운 컬럼 생성	기존 컬럼 값 수정
• 새로운 컬럼 Age_0 생성 • 모든 행이 0 값이 일괄 적용 상수 값 넣으면 모든 데이터 동일 적용	• Age_by_10 : Age x 10 • Family_No : 형제 + 부모+ 본인 기존 데이터를 가공해서 새로운 컬럼 생성 가능	• 기존 컬럼 값이 10을 더해 업데이트
<code>df['Age_0'] = 0</code>	<code>df['Age_by_10'] = df['Age'] * 10</code>	<code>df['Age_by_10'] = df['Age_by_10'] + 10</code>

DataFrame의 데이터 삭제

drop() 메서드를 사용

DataFrame.drop(labels, axis, inplace)

파라미터	설명
<code>labels</code>	삭제할 행 또는 컬럼 이름
<code>axis</code>	0 = 행 삭제 / 1 = 컬럼 삭제
<code>inplace</code>	True: 원본 데이터 수정 / False: 결과만 반환

DataFrame의 데이터 삭제- 컬럼 삭제

```
df.drop('Age_0', axis=1)
```

-> axis = 1: 컬럼 삭제

```
df.drop(['Age_0', 'Age_by_10', 'Family_No'],  
axis=1)
```

-> 여러 컬럼 삭제

DataFrame의 데이터 삭제- 행 삭제

```
df.drop([0,1,2], axis=0)
```

-> axis = 0: 행 삭제

*인덱스 번호 기준으로 삭제

DataFrame의 데이터 삭제- inplace

```
df.drop('Age_0', axis=1, inplace=True)
```

True: 원본 DataFrame 변경

False(기본값): 삭제된 결과만 반환

01

index개념

Dataframe 각 행을 식별하는 객체
데이터의 고유 식별자 역할

02

index특징

실제 값은 numpy 1차원 배열 형태임
슬라이싱 가능/ 값 변경 불가능

연산 특성

index는 연산에서 제외됨

*데이터의 값만 연산하고 index는 식별 역할만 수행

index재생성

`df.reset_index()`

기존 index는 컬럼으로 저장/ 새로운 연속 숫자 index생성

index 객체

[Index 추출]

```
Indexes = df.index  
print(indexes)
```

[슬라이싱]

```
Indexes[:5]  
indexes[6]
```



[값 변경 불가]

~~Indexes[0] = 5~~

데이터 선택 및 필터링

01 []

컬럼 선택/ 조건 필터링 도구

02 iloc[]

위치 기반 인덱싱

03 loc[]

이름 기반 인덱싱

04 boolean

조건 필터링

데이터 선택 및 필터링

01 []

컬럼 선택/ 조건 필터링 도구

02 iloc[]

위치 기반 인덱싱

03 loc[]

이름 기반 인덱싱

04 boolean

조건 필터링

데이터 선택 및 필터링 - DataFrame[]

df['Pclass']	#단일컬럼
df[['Survived' , 'Pclass']]	#여러컬럼

df[0]	#숫자인덱스 사용 불가
-------	--------------

데이터 선택/ 필터링 도구

숫자 인덱스 사용 불가

데이터 셀렉션 및 필터링 - iloc[]

위치 기반 인덱싱
행 위치, 열 위치 사용

- 정수만 사용 가능
- Boolean 인덱싱 불가

```
df.iloc[0,0]  
# 첫 행 첫 열
```

```
df.iloc[0:2,0:3]  
# 슬라이싱
```

데이터 셀렉션 및 필터링 - loc[]

이름 기반 인덱싱

행: index 이름, 열: column 이름

- 슬라이싱 시 끝 값 포함

```
df.loc['one', 'Name']  
df.loc['one':'three', 'Name':'Gender']
```

데이터 선택 및 필터링 - boolean

조건 필터링

df[조건] 형태로 사용

```
df[df['Age'] > 60]
```

```
df[df['Age'] > 60][['Name', 'Age']]
```

```
df.loc[df['Age'] > 60, ['Name', 'Age']]
```


정렬, aggregation 함수, groupby

1. 정렬 `sort_values()`: 데이터를 특정 컬럼 기준으로 정렬

2. Aggregation 함수: 데이터 통계값 계산

`count()`

`sum()`

`mean()`

`max()`

`min()`

3. GroupBy: 특정 기준으로 데이터를 그룹화 후 통계 계산

pandas 데이터 처리- 결손 확인

결손 데이터(missing data)

결손 데이터: 값이 없는 데이터 -> NaN으로 표시
머신러닝 모델은 NaN처리 x -> 대체 필요

```
df.isna()  
df.isna().sum()
```

True = 1, False = 0 -> sum()으로 개수 확인

pandas 데이터 처리- 결손 처리

fillna()

1. 특정 값으로 대체
2. 평균 값으로 대체

3.

```
df['Cabin'] = df['Cabin'].fillna('C000')  
df['Embarked'] = df['Embarked'].fillna('S')
```


apply + lambda 데이터 가공

1. lambda: 한 줄로 만든 간단한 함수
ex. `lambda x: x**2`

2. `apply( )`: DataFrame 또는 Series의 각 데이터에 함수 적용

```
df['NameL'] = df['Name'].apply(lambda x: len(x))
```

Name 컬럼의 각 문자열을 `lambda x: len(x)` 로 처리하여 이름의 문자열 길이를 계산하고 그 결과를 새로운 컬럼 NameL에 저장함

THANK YOU

